

Sentiment Analysis Report: Comparison of Models on Alocine Dataset

Ryhem Mekni and Jafra Chairi

September 28, 2025

1 Introduction

This project aims to predict whether French movie reviews from the Alocine dataset are positive or negative. We implemented three models: (1) a baseline model using TF-IDF with Logistic Regression and LinearSVC, (2) a Bidirectional LSTM (BiLSTM) with learned embeddings, and (3) a BiLSTM with a Bahdanau attention mechanism. The models were evaluated on accuracy, precision, recall, and F1-score, with a focus on comparing performance and interpreting attention weights. This report details the code implementation, performance results, and insights from training dynamics and attention weights.

2 Dataset

The Alocine dataset, sourced from [Hugging Face](#), contains French movie reviews labeled as positive (1) or negative (0). The dataset is split into:

- Training set: 160,000 reviews
- Validation set: 20,000 reviews
- Test set: 20,000 reviews

2.1 Preprocessing

- Text Cleaning: A `clean_text` function converted text to lowercase, removed extra whitespace, and stripped punctuation. `IDF`: Text was transformed into TF-IDF vectors with `max_features = 50000`, `ngram_range = (1, 2)`, `min_df = 5`, and `strip_accents = 'unicode'`.
- Tokenization and Padding: Reviews were tokenized with a 50,000-word vocabulary and padded to 200 tokens.

3 Model Implementations

3.1 Baseline Models: TF-IDF + Logistic Regression and LinearSVC

Overview: These models use TF-IDF to transform text into numerical vectors, followed by Logistic Regression or LinearSVC.

Code Explanation:

- TF-IDF Vectorization: Configured with `max_features = 50000`, `ngram_range = (1,2)`, `min_df = 5`, and `strip_accents = 'unicode'`. *LogisticRegression : Used `max_iter = 1000`, `solver = 'saga'`, and `n_jobs = -1`.*
- LinearSVC: Configured with `max_iter = 5000`.

```
tfidf = TfidfVectorizer(max_features=50000, ngram_range=(1,2),
    min_df=5, strip_accents='unicode')
lr = LogisticRegression(max_iter=1000, n_jobs=-1, solver="saga")
pipe_lr = Pipeline([("tfidf", tfidf), ("clf", lr)])
pipe_lr.fit(train_df["text"], y_train)
```

3.2 BiLSTM Model with Embeddings

Overview: A BiLSTM with learned embeddings captures sequential patterns.

Code Explanation:

- Tokenization and Padding: Texts were tokenized and padded to 200 tokens.
- Architecture: Includes an Embedding layer (50,000 input, 100 output), a Bidirectional LSTM (128 units), two Dropout layers (0.3), a Dense layer (64 units, ReLU), and a final Dense layer (sigmoid).
- Training: Compiled with `binary_crossentropy` loss and `Adam` optimizer, trained for 8 epochs with Early Stop.

```
MAX_VOCAB = 50000
MAX_LEN = 200
EMBED_DIM = 100
tokenizer = Tokenizer(num_words=MAX_VOCAB, oov_token="<OOV>")
tokenizer.fit_on_texts(train_df["text"].tolist())
X_train_seq = pad_sequences(tokenizer.texts_to_sequences(train_df["text"].tolist()), maxlen=MAX_LEN, padding="post", truncating="post")

def build_rnn_model(vocab_size=MAX_VOCAB, embed_dim=EMBED_DIM,
    input_len=MAX_LEN, rnn_units=128, dropout=0.3):
    inp = layers.Input(shape=(input_len,))
    x = layers.Embedding(input_dim=vocab_size, output_dim=
        embed_dim, input_length=input_len)(inp)
    x = layers.Bidirectional(layers.LSTM(rnn_units,
        return_sequences=False))(x)
    x = layers.Dropout(dropout)(x)
    x = layers.Dense(64, activation="relu")(x)
```

```

x = layers.Dropout(dropout)(x)
out = layers.Dense(1, activation="sigmoid")(x)
model = models.Model(inputs=inp, outputs=out)
model.compile(loss="binary_crossentropy", optimizer=
    optimizers.Adam(1e-3), metrics=["accuracy"])
return model

rnn_model = build_rnn_model()
history_rnn = rnn_model.fit(X_train_seq, y_train, validation_data
    =(X_val_seq, y_val), epochs=8, batch_size=256, callbacks=[
    callbacks.EarlyStopping(monitor="val_loss", patience=3,
        restore_best_weights=True),
    callbacks.ModelCheckpoint("rnn_model.h5", save_best_only=True
        , monitor="val_loss")
])

```

3.3 BiLSTM with Bahdanau Attention

Overview: Extends the BiLSTM with a Bahdanau attention layer for focus on important words.

Code Explanation:

- Bahdanau Attention: A custom layer computes attention scores and weights.
- Architecture: Includes an Embedding layer (50,000 input, 100 output), a Bidirectional LSTM (128 units, `return_sequences = True`), *GlobalAveragePooling1D* for the query, *BahdanauAttention*. Similar to the BiLSTM model.

```

class BahdanauAttention(Layer):
    def __init__(self, units):
        super(BahdanauAttention, self).__init__()
        self.W1 = layers.Dense(units)
        self.W2 = layers.Dense(units)
        self.V = layers.Dense(1)

    def call(self, query, values):
        query_with_time_axis = tf.expand_dims(query, 1)
        score = self.V(tf.nn.tanh(self.W1(query_with_time_axis) +
            self.W2(values)))
        attention_weights = tf.nn.softmax(score, axis=1)
        context_vector = attention_weights * values
        context_vector = tf.reduce_sum(context_vector, axis=1)
        return context_vector, tf.squeeze(attention_weights, -1)

def build_rnn_with_attention(vocab_size=MAX_VOCAB, embed_dim=
    EMBED_DIM, input_len=MAX_LEN, rnn_units=128):
    inp = layers.Input(shape=(input_len,))
    x = layers.Embedding(vocab_size, embed_dim, input_length=
        input_len)(inp)
    lstm_out = layers.Bidirectional(layers.LSTM(rnn_units,
        return_sequences=True))(x)

```

```

query = layers.GlobalAveragePooling1D()(lstm_out)
attention = BahdanauAttention(units=64)
context_vector, att_weights = attention(query, lstm_out)
x = layers.Dropout(0.3)(context_vector)
x = layers.Dense(64, activation="relu")(x)
x = layers.Dropout(0.3)(x)
out = layers.Dense(1, activation="sigmoid")(x)
model = models.Model(inputs=inp, outputs=out)
model.compile(loss="binary_crossentropy", optimizer=
    optimizers.Adam(1e-3), metrics=["accuracy"])
return model

att_model = build_rnn_with_attention()
history_att = att_model.fit(X_train_seq, y_train, validation_data
    =(X_val_seq, y_val), epochs=8, batch_size=256, callbacks=[
    callbacks.EarlyStopping(monitor="val_loss", patience=3,
        restore_best_weights=True),
    callbacks.ModelCheckpoint("att_model.h5", save_best_only=True
        , monitor="val_loss")
])

```

	Layer (type)	Output Shape
Model Summary:	$input_{layer_1}$ (<i>InputLayer</i>)	(None, 200)
	$embedding_1$ (<i>Embedding</i>)	(None, 200, 100)
	$bidirectional_1$ (<i>Bidirectional</i>)	(None, 200, 256)
	$global_{average_pooling1d}$ (<i>GlobalAveragePooling1D</i>)	(None, 256)
	$bahdanau_{attention}$ (<i>BahdanauAttention</i>)	[(None, 256), (None, 200)]
	$dropout_2$ (<i>Dropout</i>)	(None, 256)
	$dense_5$ (<i>Dense</i>)	(None, 64)
	$dropout_3$ (<i>Dropout</i>)	(None, 64)
	$dense_6$ (<i>Dense</i>)	(None, 1)

4 Performance Comparison

The models were evaluated on the test set. Results are:

Model	Accuracy	Precision	Recall	F1-Score
TF-IDF + Logistic Regression	0.9406	0.9354	0.9412	0.9383
TF-IDF + LinearSVC	0.9430	0.9368	0.9450	0.9408
BiLSTM with Embeddings	0.9353	0.9343	0.9305	0.9324
BiLSTM with Attention	0.9375	0.9289	0.9417	0.9353

Analysis:

- TF-IDF + LinearSVC leads with 0.9430 accuracy and 0.9408 F1-score.
- BiLSTM with Embeddings (0.9353 accuracy) is slightly lower, possibly due to overfitting.
- BiLSTM with Attention (0.9375 accuracy) improves recall (0.9417), indicating better detection of positive reviews.

5 Interpretation of Attention Weights

The Bahdanau attention layer assigns weights to each token, highlighting important words.

- Extraction: Returns `att_weights(shape : None, 200)`. *Visualization : Suggested code :*

```
tokens, weights = get_attention_weights(att_model, X_test_eq, tokenizer, index = 0)
plt.bar(tokens[:20], weights[:20])
plt.xticks(rotation = 45)
plt.title("AttentionWeights")
plt.show()
```

- Example: For "Ce film est absolument fantastique," "fantastique" and "absolument" likely have high weights.

Insights: Focuses on sentiment-laden terms, enhancing interpretability.

6 Training Dynamics

Training and validation loss/accuracy were visualized using `plot_history`. The plots show :

- **RNN Model:**
 - Loss: Training loss decreases from 0.25 to 0.10, validation loss stabilizes at 0.15 after 2 epochs, indicating early convergence with some overfitting.
 - Accuracy: Training accuracy rises to 0.98, validation accuracy stabilizes at 0.94.
- **Attention RNN Model:**
 - Loss: Training loss decreases to 0.10, validation loss stabilizes at 0.15 after 2 epochs, with similar overfitting.
 - Accuracy: Training accuracy reaches 0.98, validation accuracy stabilizes at 0.94.

```
def plot_history(h, title="history"):
    plt.figure(figsize=(12,4))
    plt.subplot(1,2,1)
    plt.plot(h.history["loss"], label="train_loss")
    plt.plot(h.history["val_loss"], label="val_loss")
    plt.title(title + "┐┐loss"); plt.legend()
    plt.subplot(1,2,2)
    plt.plot(h.history["accuracy"], label="train_acc")
    plt.plot(h.history["val_accuracy"], label="val_acc")
    plt.title(title + "┐┐acc"); plt.legend()
    plt.show()

plot_history(history_rnn, "RNN")
plot_history(history_att, "Attention┐RNN")
```

Observations: Both models converge within 2-3 epochs, with EarlyStopping triggering around epoch 4-5. The slight increase in validation loss suggests overfitting, mitigated by dropout (0.3).

7 Conclusion

The TF-IDF + LinearSVC model achieved the highest performance (0.9430 accuracy, 0.9408 F1-score). The BiLSTM with attention (0.9375 accuracy, 0.9353 F1-score) offers improved recall and interpretability. Training plots indicate convergence within 2-3 epochs, with some overfitting. Future work could explore pre-trained embeddings or transformer models like BERT.

Recommendations: Deploy LinearSVC for optimal metrics, but use the attention model for explainability. Adjust hyperparameters or increase data to reduce overfitting.